Universidad Autónoma de Santo Domingo
Primada de América / Fundada el 28 de octubre de 1538

Unidad numero III, Diagrama UML

100738271 Jeffry Gilma

INF5120-3 Lenguaje de programacion I

Víctor Núñez Vicente

Peravia: República Dominicana

3 Marzo 2026

Describa brevemente el objetivo del sistema desarrollado y el contexto del problema.

El sistema desarrollado tiene como objetivo gestionar productos dentro de una empresa comercial, permitiendo registrarlos, buscarlos, eliminarlos y listarlos de manera organizada. También controla que no existan códigos duplicados y muestra la cantidad total de productos almacenados.

El problema surge porque la empresa necesita una herramienta básica para administrar su inventario de forma estructurada y eficiente. Por ello, se desarrolló una aplicación en C# aplicando principios de Programación Orientada a Objetos y utilizando un contenedor adecuado para almacenar los productos.

Análisis técnico

1. Tipo de contenedor utilizado

En este programa use array

Porque?
Porque Permite entender mejor el manejo manual de memoria, ademas que Obliga a controlar el tamaño y desplazamiento de elementos. Cumple correctamente los requisitos del sistema, Es más fundamental y muestra dominio de estructuras básicas.

Comparación:

Array

Criterio:
Tamaño dinámico: Fijo
-Facilidad de uso: Mas complejo
Rendimiento: Ligeramente mas rápido en acceso directo
Complejidad de código: Mayor, osea que requiere moverse mas lento

List <T>

Criterio: Dinámico
Tamaño dinámico: Mas simple
Facilidad de uso: Muy eficiente, Ligera

Rendimiento: sobrecarga
Complejidad de código: Menor (métodos add, remuve, etc.)

El Array requiere mayor control manual mientras que el List<T> es mas flexible y recomendado en sistemas reales.

3. Diseño Orientado a Objetos

¿Qué tipo de relación existe entre Almacén y Producto?

Una relación de composición

Porque?
Porque el almacén contiene múltiples objetos productos y los productos existen dentro del almacén.

¿Qué ocurriría si el Almacén se elimina?

Si el objeto almacén se destruye, los productos contenidos también desaparecen, ya que dependen estructuralmente del almacén.

Diagrama UML- Sistema de gestión de productos

Clase: Producto

código

Producto	
-Código: -Nombre: -Presio:	String string double
+ Producto (c,n,p)	

Tiene tres atribbutos publicos, cuenta cin un conductor para iniciar los datos y es una clase simple de entidad (modelo)

Clase: Almacén

Almacén
-productos : producto [] -cantidad : int
+Almacén (cantidad : int) +Agregarproducto (…) : bool

+Buscarproducto (codigo) : producto +Eliminarproducto (codigo) : bool +Listarproductos () : void +Obtenercantidad () : int

Esta clase contiene un arreglo de objetos “producto” y que controla la cantidad de productos registrados, ademas que gestiona todas las operaciones del sistema.

Relación entre clases

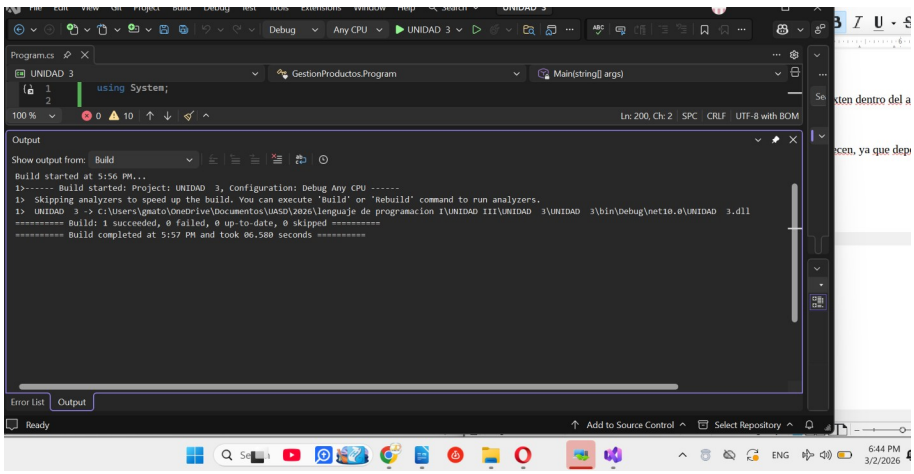
Almacén <--------- producto
(1) (0...*)

Tiene un tipo de relación composición. Donde el almacén contiene productos, si el almacén se elimina, los productos se eliminan, osea que los productos dependen del almacén.

Existe una relación de composición entre almacén y producto, ya que el almacén esta compuesto por múltiples productos y estos no tienen sentido dentro del sistema si el almacén deja de existir. La vida de los objetos producto depende directamente del objeto almacén

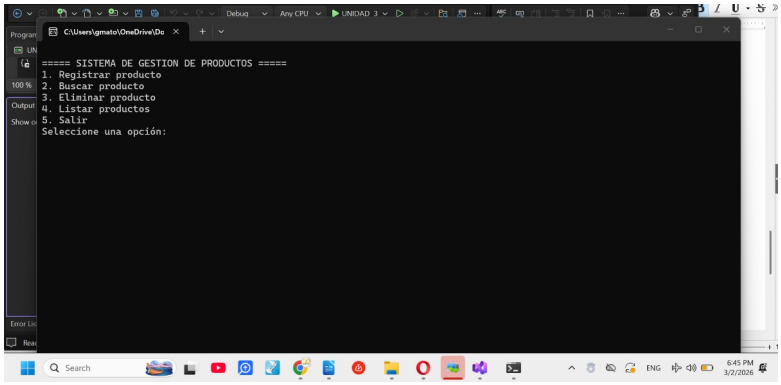
Evidencia de ejecución

output de ejection:

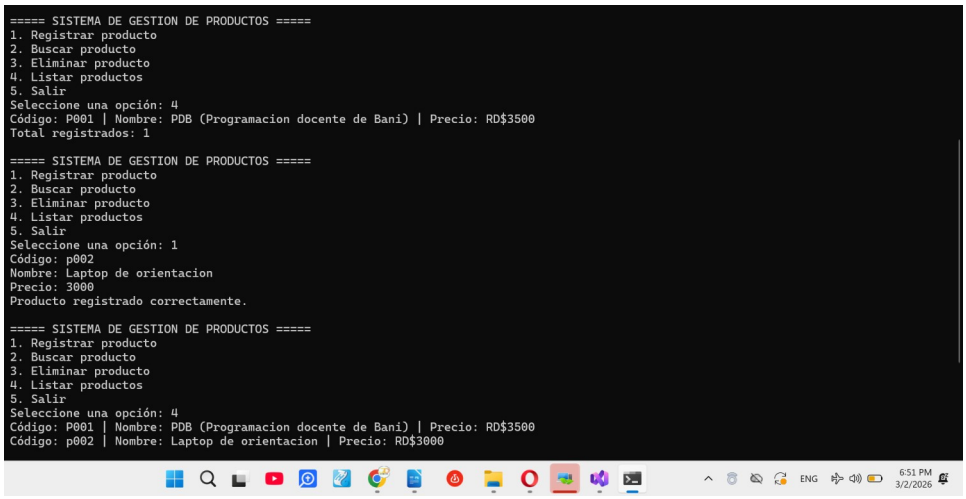


Output terminal:

1.

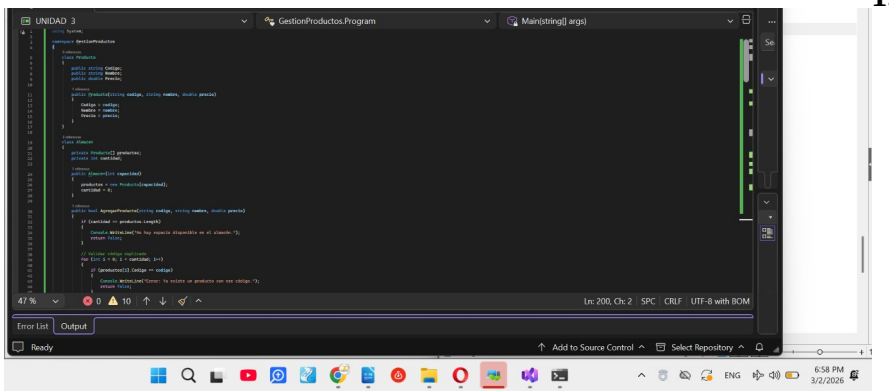


2.

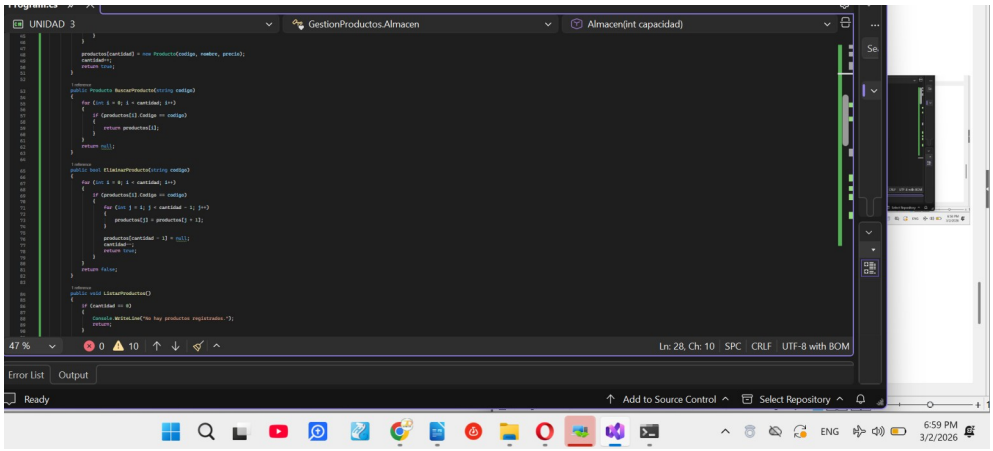


Codigos del Programa

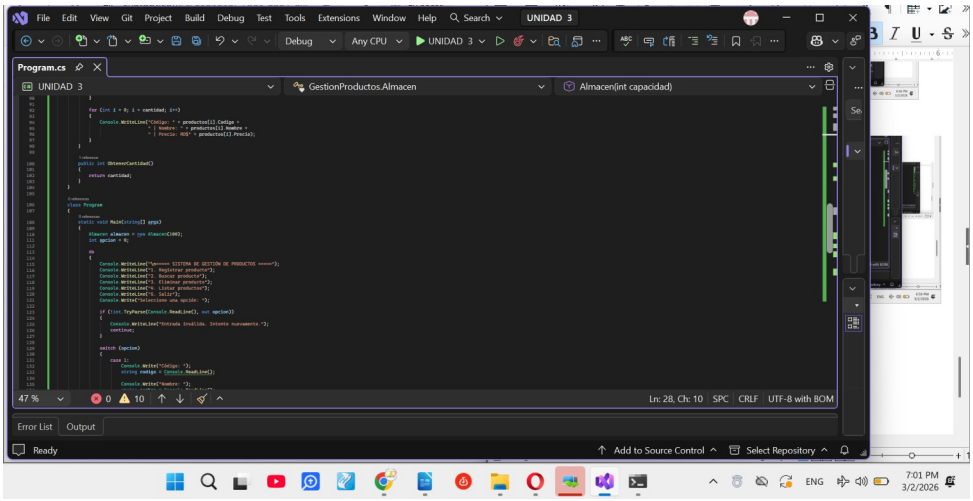
1.



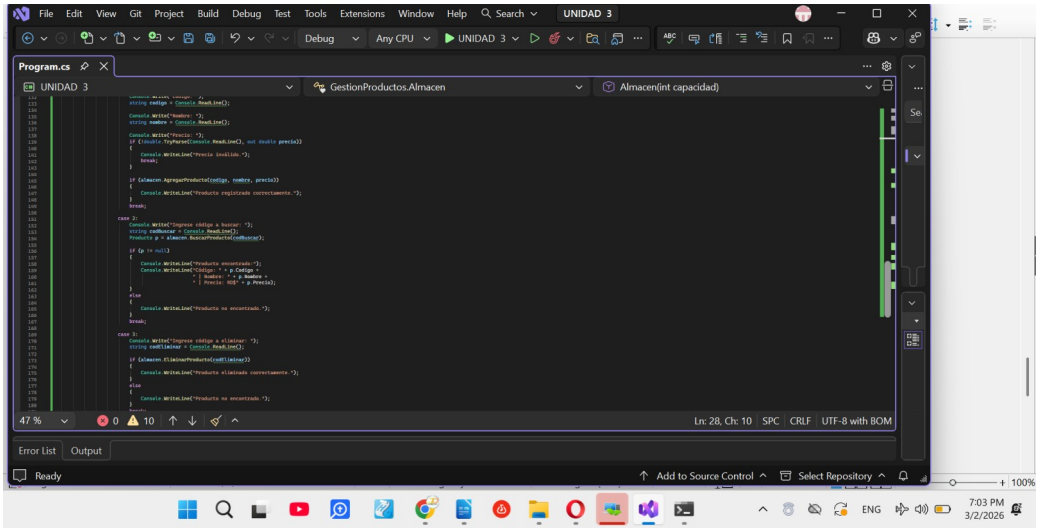
2.


UNIDAD 3
GestionProductos.Almacen
Almacen(int capacidad)
Ln: 28, Ch: 10
SPC
CRLF
UTF-8 with BOM
Error List
Output
Ready
Add to Source Control
Select Repository
6:59 PM
3/2/2026

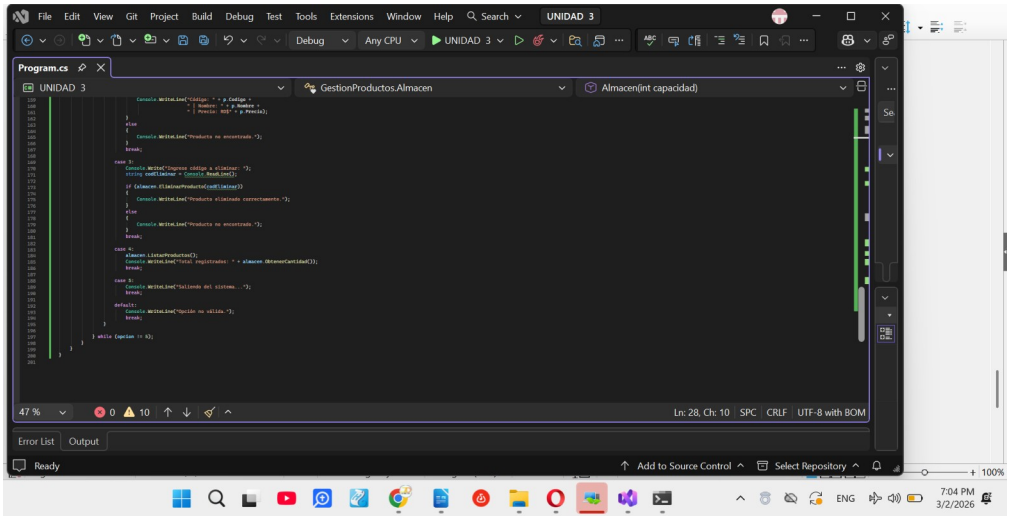
3.


File
Edit
View
Git
Project
Build
Debug
Test
Tools
Extensions
Window
Help
Search
UNIDAD 3
Debug
Any CPU
Program.cs
GestionProductos.Almacen
Almacen(int capacidad)
Ln: 28, Ch: 10
SPC
CRLF
UTF-8 with BOM
Error List
Output
Ready
Add to Source Control
Select Repository
7:01 PM
3/2/2026

4.


File
Edit
View
Git
Project
Build
Debug
Test
Tools
Extensions
Window
Help
Search
UNIDAD 3
Debug
Any CPU
Program.cs
GestionProductos.Almacen
Almacen(int capacidad)
Ln: 28, Ch: 10
SPC
CRLF
UTF-8 with BOM
Error List
Output
Ready
Add to Source Control
Select Repository
7:03 PM
3/2/2026

5.



Análisis de los códigos:

Como códigos aritméticos, utilizamos operadores para calcular matemáticos como (*, /, %, +, -, ++, y --).

Relaciones: Utilizados para comprar valores, devolviendo un booleano, tales como “<>, <=, y >=”. Se utilizo implementación de encapsulacion, porque se solicita la relación de una clase producto que modele un objeto con integridad de datos. Atributos privados: se definen id, nombre, precio y stock.

Sistema de gestión con arreglos: se utilizo una clase de almacén, se utiliza un arreglo de objetos “producto[] productos” y un contador entero “cantidad” para rastrear cuantos espacios están ocupados.

Lógica de registro: El método “Agregarproducto” incluye validaciones criticas como verificar si el almacén esta lleno y asegura que no existan códigos de producto duplicados antes de insertar uno nuevo.

Gestión de datos (CRUD):

búsqueda: un ciclo for para iterar sobre el arreglo y comparar el código ingresado con los existentes
.

Interfaz de usuario main: implementa un ciclo “do-while” con un menu swith, permitiendo al usuario interactuar repetidamente con el sistema hasta elegir la opción de salida.

Concepto de escalas

Fuera del ámbito de la programación pura, este documento explica el cálculo de escalas para gráficas físicas.

Formula General: escala = [valor mayor / espacio disponible].

Conclusion

En esta unidad aprendí a aplicar los principios de la Programación Orientada a Objetos en el desarrollo de un sistema funcional, comprendiendo mejor la importancia de la organización del código mediante clases, atributos y métodos. También reforcé el uso de estructuras de almacenamiento como arreglos y colecciones genéricas, analizando sus diferencias y ventajas según el contexto del problema.

Una de las principales dificultades fue decidir qué tipo de contenedor utilizar y justificar técnicamente esa elección, ya que implicó comparar aspectos como tamaño dinámico, rendimiento y complejidad del código. Además, manejar correctamente la validación de datos y evitar errores en tiempo de ejecución requirió mayor atención y práctica.

Sin embargo, estas dificultades permitieron fortalecer mi comprensión sobre el diseño estructurado de programas y la importancia de escribir código claro, funcional y mantenible.

Declaración de originalidad: Este documento es de mi autoría y cumple con las normas académicas establecidas.